

Итоговый проект

Вариант второй

Интерактивный CLI-трекер привычек (HabitHero)

Цель проекта: Разработать консольное приложение для формирования и отслеживания привычек с возможностью настройки индивидуального гибкого графика для каждой из них. Программа должна мотивировать пользователя с помощью системы страйков. За выполнение задач начисляется опыт, а за пропуски расчетных дней - цепочки сгорают.

Архитектура и структура классов

Класс Habit

Описывает одну конкретную привычку с её уникальным расписанием.

Атрибуты:

- title (str): Название привычки.
- description (str): Короткое описание/цель.
- category (str): Категория привычки ("Спорт", "Интеллект" или "Быт").
- schedule_type (str): Тип расписания ("everyday" - каждый день, "custom" - по выбранным дням недели).
- schedule_days (list[int]): Список дней недели для типа custom, где 0 - Понедельник, 2 - Среда и т.д. (пустой список для everyday).
- creation_date (datetime.date): Дата создания привычки.
- dates_completed (list[datetime.date]): Список всех дат успешного выполнения.
- current_streak (int): Текущая непрерывная цепочка выполнения (учитывает график расписания).
- max_streak (int): Исторический рекорд (максимальная цепочка) этой привычки.

Методы:

- check_in(date=None): Отмечает выполнение привычки. Если дата не передана, используется сегодняшняя. Добавляет её в dates_completed и обновляет страйки.
- to_dict() -> dict: Сериализует объект привычки в словарь для JSON (конвертируя объекты дат в строки YYYY-MM-DD).

ИТОГОВЫЙ проект

Вариант второй

Интерактивный CLI-трекер
привычек (HabitHero)

Архитектура и структура классов

Класс HabitManager

Ядро системы, управляющее всеми процессами. Внутри класса запрещены функции `input()` и `print()`.

Атрибуты:

- `user (UserCharacter)`: Объект игрового персонажа.
- `habits (list[Habit])`: Список всех отслеживаемых объектов привычек.

Методы:

- `add_habit(title: str, description: str, category: str, schedule_type: str, schedule_days: list[int] = None)`: Создает и добавляет новую привычку.
- `delete_habit(title: str)`: Удаляет привычку по названию.
- `complete_habit(title: str) -> dict`: Находит 習慣, вызывает `check_in()`, начисляет персонажу опыт (базовый + бонус за страйк), проверяет, побит ли рекорд, и возвращает словарь с результатами для CLI.
- `update_streaks_and_penalties() -> list[str]`: Запускается автоматически при старте программы. Проверяет пропущенные дни по расписанию, сбрасывает страйки нарушителям, начисляет штрафы персонажу и возвращает список логов нарушений.

Класс UserCharacter

Отвечает за геймификацию, уровни и профиль пользователя.

Атрибуты:

- `username (str)`: Имя пользователя.
- `level (int)`: Общий уровень персонажа (начинается с 1).
- `experience (int)`: Текущие очки общего опыта.
- `skills (dict[str, int])`: Уровни профильных навыков по категориям (например: {"Спорт": 1, "Интеллект": 1, "Быт": 1}).

Методы:

- `add_xp(amount: int, category: str) -> bool`: Добавляет опыт. Опыт капает в общую шкалу и развивает навык указанной категории. При достижении порога общего опыта ($level * 100$) повышает `level` и возвращает `True` (сигнал интерфейсу для Level Up).
- `deduct_xp(amount: int)`: Снижает общий опыт за пропуски дней в качестве штрафа (опыт не может упасть ниже 0).
- `to_dict() -> dict`: Сериализует профиль игрока в словарь для JSON.

Итоговый проект

Вариант второй

Интерактивный CLI-трекер
привычек (HabitHero)

Механика «Сгорания страйков» при гибком расписании

Проверка пропущенных дней усложняется из-за наличия индивидуальных графиков. Алгоритм запускается синхронно при старте программы.

- Для ежедневных привычек (everyday): Если последняя дата выполнения была позавчера или раньше - страйк сгорает (`current_streak = 0`), персонаж штрафуются на 15 XP, а система генерирует уведомление.
- Для гибких привычек (custom): Программа не может просто проверить вчерашний день. Она должна определить, какой день согласно расписанию этой привычки был последним расчетным днем перед сегодняшним.
 - Пример: Если привычка стоит по графику «Понедельник, Среда», а сегодня Пятница. Последним расчетным днем была Среда. Если в Среду отметки не было - страйк сгорает. Если в Среду отметка была, а сегодня Пятница и день еще не закончился - страйк удерживается, а статус привычки на сегодня: [] (Не выполнено).
- Если расчетный день пропущен: `current_streak` сбрасывается в 0, персонаж теряет 15 XP, генерируется лог: «Привычка [Название] прервана! Расчетный день ([День недели]) был пропущен. Страйк сброшен!».

Итоговый проект

Вариант второй

Интерактивный CLI-трекер
привычек (HabitHero)

Расчет исторического рекорда

Программа должна мотивировать пользователя побить свои же достижения.

- При каждом успешном выполнении привычки (`check_in`), когда `current_streak` увеличивается, класс должен сравнить его с `max_streak`.
- Если текущий страйк стал больше максимального, `max_streak` обновляется, а метод сообщает интерфейсу, что зафиксирован новый личный рекорд.

CLI-визуализация: Календарная сетка прогресса

Для демонстрации успехов на финальном занятии студенты должны написать алгоритм текстовой визуализации истории за последние 7 дней.

- Логика работы: Программа берет текущую дату и 6 предыдущих дней. Для каждой привычки она проверяет, содержатся ли эти даты в `dates_completed`.

Итоговый проект

Вариант второй

Интерактивный CLI-трекер
привычек (HabitHero)

Логика работы с хранилищем

Вся база хранится в файле habits.json. Структура файла должна поддерживать вложенность (данные пользователя + список привычек со списками дат):

```
{
  "user": {
    "username": "Hero123",
    "level": 3,
    "experience": 240,
    "skills": {
      "Спорт": 2,
      "Интеллект": 1,
      "Быт": 3
    }
  },
  "habits": [
    {
      "title": "Сделать зарядку",
      "description": "Утренняя разминка",
      "category": "Спорт",
      "schedule_type": "custom",
      "schedule_days": [0, 2, 4],
      "creation_date": "2026-06-01",
      "current_streak": 5,
      "max_streak": 12,
      "dates_completed": ["2026-06-24", "2026-06-26",
"2026-06-29"]
    }
  ]
}
```


Итоговый проект

Вариант второй

Интерактивный CLI-трекер
привычек (HabitHero)

Интерфейс (CLI) и логика взаимодействия

Программа работает в бесконечном цикле текстового меню, защищенном блоками try-except.

Главное меню:

1. Мой профиль и привычки: Выводит статус персонажа (уровень, опыт) и интерактивный список привычек с их страйками и историей за неделю.
2. Отметить выполнение: Выводит список невыполненных на сегодня привычек, пользователь вводит номер, привычка отмечается, начисляется опыт. Если произошёл Level Up, выводится красивое текстовое поздравление.
3. Создать новую привычку: Запрос названия и описания.
4. Удалить привычку: Исключение привычки из трекера.
5. Выход: Сохранение в JSON и закрытие программы.

Итоговый проект

Вариант второй

Интерактивный CLI-трекер
привычек (HabitHero)

Критерии оценки

- Архитектурная чистота: Логика расписаний, сгорания, фильтрации дат и уровней инкапсулирована внутри классов. Интерфейсный модуль отвечает только за print и input.
- Магия дат (datetime): Безошибочный расчет пропущенных дней с учетом индивидуальных графиков (использование .weekday() и дельты дат). Корректный вывод 7-дневной сетки.
- Надежность JSON: Корректная глубокая сериализация и десериализация вложенных структур данных и дат.
- Устойчивость: Программа валидирует ввод пользователя (например, не позволяет ввести несуществующие дни недели или отрицательные числа) и не падает при ошибках.